Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas

PROYECTO 2
TaskScript

PONDERACIÓN: 40 pts

Tiempo estimado: 40 hrs

1. Marco Formativo

1.1. Valor

Nombre del Valor	¿Como se aplica en tu laboratorio?
Perseverancia	El análisis sintáctico requiere diseñar, probar y corregir gramáticas iterativamente. Los estudiantes desarrollan perseverancia al enfrentar errores de parsing, refinar producciones gramaticales y depurar arboles de derivación hasta lograr un analizador funcional.

1.2. Competencia(s)

Tipo de Competencia	Descripción
Competencia General	Diseñar, implementar y evaluar soluciones computacionales aplicando los fundamentos teóricos y prácticos de las ciencias de la computación para resolver problemas reales en el ámbito del desarrollo de software.
Competencia Especifica	Generar gramáticas libres de contexto y aplicar análisis sintáctico para reconocer y validar la estructura de un lenguaje de dominio especifico, implementando un parser manual en C++ que construya un árbol de derivación a partir de los tokens reconocidos por el analizador léxico.

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Especifico (¿Qué?)	El objetivo es concreto y tangible.	Desarrollar una aplicación de escritorio en C++ con interfaz gráfica (Qt o wxWidgets) que implemente un analizador léxico y sintáctico capaz de interpretar archivos .task con estructura de gestión de tareas tipo Kanban, generando reportes HTML y un diagrama Graphviz del árbol de derivación.
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	El analizador debe reconocer correctamente al menos el 90% de los tokens y validar la estructura sintáctica del lenguaje TaskScript, detectar errores sintácticos con posición exacta, generar 3 reportes HTML y el árbol de derivación en formato Graphviz.
Alcanzable (¿Como?)	Posible con los recursos disponibles.	Utilizando C++17 con STL, implementando el AFD manualmente para el lexer y un parser descendente recursivo para el análisis sintáctico, usando Qt 5/6 o wxWidgets para la GUI.
Realista (Para qué?)	Contribuye a metas más amplias.	Fortalecer competencias en gramáticas libres de contexto, análisis sintáctico, diseño de parsers y generación de reportes, preparando al estudiante para los cursos de Compiladores 1 y 2.

A Tiempo (Cuando?)	Tiene fecha límite definida.	Completar el proyecto en 4 semanas: Semana 1 diseño de la gramática y tokens, Semana 2 implementación del lexer y parser, Semana 3 GUI y reportes, Semana 4 pruebas y documentación.

2. Resumen Ejecutivo

TaskScript es un proyecto de desarrollo de software cuyo objetivo es construir una herramienta de escritorio en C++ que permita analizar léxica y sintácticamente archivos de texto con extensión .task, estructurados bajo las reglas del mini-lenguaje TaskScript. Este lenguaje de dominio especifico está orientado a la gestión de tareas tipo Kanban, permitiendo definir tableros con columnas y tarjetas que contienen atributos como prioridad, responsable y fecha límite.

El proyecto se enfoca en dos fases del pipeline de compilación: el análisis léxico (implementado con un AFD manual, sin std::regex) y el análisis sintáctico (implementado con un parser descendente recursivo basado en una gramática libre de contexto). La solución integra cinco componentes principales: el motor léxico, el parser sintáctico, la interfaz gráfica de escritorio (Qt 5/6 o wxWidgets), el generador de tres reportes HTML con información del tablero, y el módulo de visualización del árbol de derivación con Graphviz.

3. Enunciado del Proyecto

3.1 Descripción del problema a resolver

La gestión de proyectos mediante tableros Kanban es una práctica común en equipos de desarrollo de software. Herramientas como Trello, Jira y GitHub Projects permiten organizar tareas en columnas que representan estados del flujo de trabajo. Sin embargo, no existe una forma textual estandarizada para definir estos tableros que pueda ser procesada automáticamente.

El problema que este proyecto busca resolver es la necesidad de una herramienta que permita definir tableros Kanban en archivos de texto plano, validar su estructura léxica y sintáctica, y generar reportes visuales útiles para la gestión del proyecto. La herramienta debe interpretar archivos .task escritos en el mini-lenguaje TaskScript, detectar y reportar errores léxicos y sintácticos con precisión, y generar reportes HTML que faciliten la visualización del estado del tablero.

3.2 Alcance del proyecto

Alcance obligatorio:

- Lenguaje TaskScript con tokens: palabras reservadas (TABLERO, COLUMNA, tarea, prioridad, responsable, fecha_limite), enumeraciones de prioridad (ALTA, MEDIA, BAJA), literales de fecha (AAAA-MM-DD), cadenas de texto, enteros y delimitadores estructurales.
- AFD implementado manualmente en C++17 con la clase LexicalAnalyzer y el método nextToken(). Prohibido el uso de std::regex para la tokenizacion principal.

- Parser descendente recursivo implementado manualmente en C++ con la clase SyntaxAnalyzer. Prohibido el uso de herramientas de generación de parsers (ANTLR, Bison, Yacc, etc.).
- Tabla de tokens: numero, lexema, tipo de token, línea y columna con precisión del 100%.
- Tabla de errores léxicos y sintácticos: numero, lexema/token, tipo de error, descripción y posición exacta. El análisis debe continuar tras cada error sin detenerse.
- Tres reportes HTML con CSS embebido (descritos en detalle en la sección 3.5).
- Diagrama Graphviz (código DOT) con el árbol de derivación resultante del análisis sintáctico.
- Interfaz gráfica en Qt 5/6 o wxWidgets 3.x con todos los paneles requeridos.

Alcance opcional:

- Resaltado de sintaxis en tiempo real en la GUI.
- Exportación de la tabla de tokens a CSV o JSON.
- Detección automática de tareas vencidas basado en la fecha actual.
- Estadísticas del análisis: frecuencia de tokens por tipo, tiempo de análisis en ms.
- Soporte para analizar múltiples archivos .task en una misma sesión.

3.3 Estructura del Lenguaje TaskScript

El archivo de entrada con extensión .task sigue una estructura jerárquica donde el elemento raíz TABLERO contiene una o más secciones COLUMNA, y cada columna contiene una o más definiciones de tarea con sus atributos.

Token / Elemento	Tipo	Descripción / Valores	Ejemplo
TABLERO	Palabra reservada raíz	Bloque principal del archivo. Sin él, el archivo es invalido.	TABLERO "Mi Proyecto" { ... }
COLUMNA	Palabra reservada de sección	Define una columna del tablero Kanban.	COLUMNA "Por Hacer" { ... }
tarea	Palabra reservada de elemento	Define una tarea con su nombre y atributos.	tarea: "Diseñar AFD" [...]
prioridad	Palabra reservada de atributo	Nivel de prioridad de la tarea.	prioridad: ALTA
responsable	Palabra reservada de atributo	Persona asignada a la tarea.	responsable: "Jorge"
fecha_limite	Palabra reservada de atributo	Fecha límite de la tarea.	fecha_limite: 2026-05-01
Prioridades	Enumeración	ALTA | MEDIA | BAJA	prioridad: MEDIA
Fecha	Literal de fecha	Formato AAAA-MM-DD (mes 01-12, día 01-31).	fecha_limite: 2026-04-15

Entero	Literal numérico	Números enteros positivos.	123
Cadena	Literal de texto	Texto entre comillas dobles.	"Implementar Lexer"
Delimitadores	Símbolos	{ } [ ] : , ;	{ , ; }

Ejemplo de archivo de entrada valido:

```
TABLERO "Proyecto LFP" {
    COLUMNA "Por Hacer" {
        tarea: "Diseñar AFD" [prioridad: ALTA, responsable: "Jorge",
            fecha_limite: 2026-05-01],
        tarea: "Implementar Lexer" [prioridad: ALTA, responsable: "María",
            fecha_limite: 2026-05-08],
        tarea: "Escribir casos de prueba" [prioridad: MEDIA,
            responsable: "Carlos", fecha_limite: 2026-05-10],
    };

    COLUMNA "En Progreso" {
        tarea: "Diseñar GUI" [prioridad: MEDIA, responsable: "Ana",
            fecha_limite: 2026-05-05],
    };

    COLUMNA "Completado" {
        tarea: "Investigar Qt" [prioridad: BAJA, responsable: "Pedro",
            fecha_limite: 2026-04-20],
        tarea: "Configurar GitHub" [prioridad: BAJA, responsable: "Jorge",
            fecha_limite: 2026-04-18],
    };
};
```

3.4 Gramática Libre de Contexto

El estudiante debe definir formalmente la gramática libre de contexto (GLC) del lenguaje TaskScript. A continuación se presenta una gramática de referencia que describe la estructura del lenguaje. El estudiante puede modificarla o extenderla con justificación en el manual técnico.

Gramática de referencia:

```
<programa>       ::= TABLERO CADENA "{" <columnas> "}" ";"

<columnas>       ::= <columna> <columnas>
                   | <columna>

<columna>        ::= COLUMNA CADENA "{" <tareas> "}" ";"

<tareas>         ::= <tarea> "," <tareas>
                   | <tarea>
```

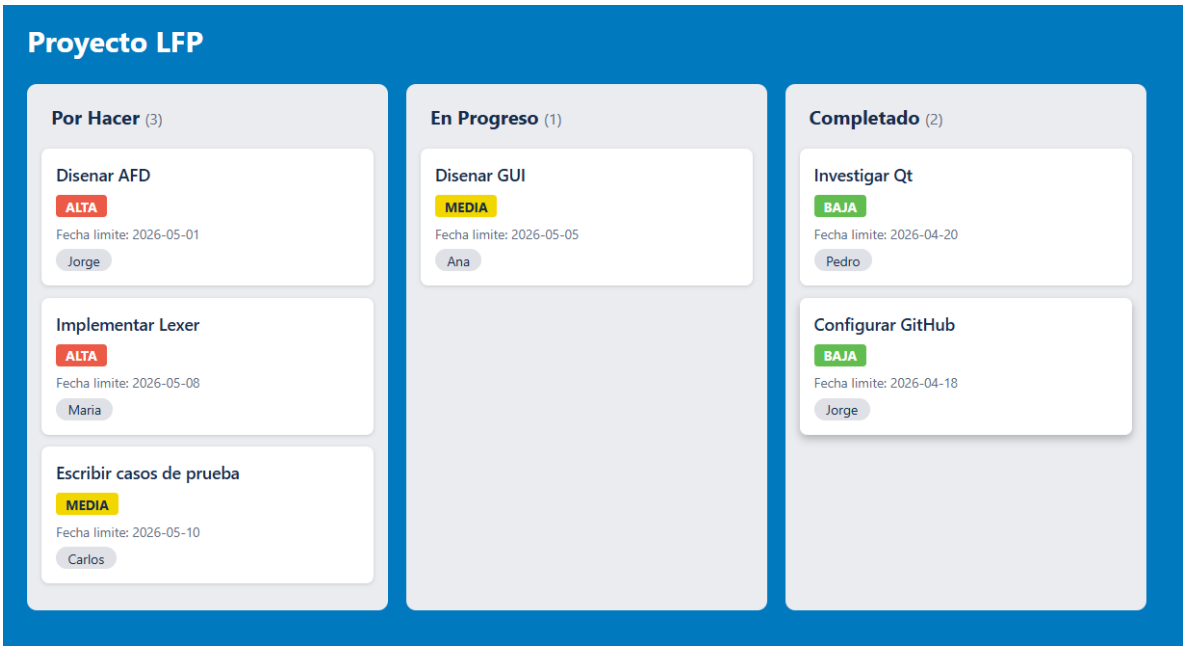
```
<tarea>          ::= tarea ":" CADENA "[" <atributos> "]"

<atributos>      ::= <atributo> "," <atributos>
                   | <atributo>

<atributo>       ::= prioridad ":" <prioridad>
                   | responsable ":" CADENA
                   | fecha_limite ":" FECHA

<prioridad>      ::= ALTA | MEDIA | BAJA
```

Nota: Esta gramática es una referencia mínima. El estudiante puede refinarla, agregar producciones para manejo de errores o extenderla para las funcionalidades opcionales. Las decisiones de diseño deben justificarse en el manual técnico.

3.5 Reportes a Generar

El sistema debe generar automáticamente tres reportes HTML al finalizar el análisis exitoso (sin errores), aprovechando la información extraída del árbol de derivación o de los tokens. Cada reporte se guarda como un archivo HTML independiente con CSS embebido, y debe poder abrirse directamente en cualquier navegador web.

Reporte 1 -- Tablero Kanban VisualObjetivo: Presentar el tablero completo en formato visual tipo Kanban, mostrando cada columna con sus tarjetas de tarea. Cada tarjeta debe mostrar el nombre de la tarea, la prioridad con color diferenciado (ALTA: rojo, MEDIA: amarillo, BAJA: verde), el responsable y la fecha límite. Las columnas se presentan lado a lado, simulando la experiencia de un tablero Trello.



Reporte 2 -- Carga por ResponsableObjetivo: Mostrar un resumen de cuantas tareas tiene asignadas cada responsable, desglosado por nivel de prioridad. Incluir una barra de progreso visual que muestre el porcentaje de tareas de cada persona sobre el total. Esto permite identificar rápidamente si hay personas sobrecargadas o subutilizadas.

Contenido mínimo requerido por fila:

- Nombre del responsable (extraído de las tareas).
- Total de tareas asignadas.
- Desglose por prioridad: cantidad de tareas ALTA, MEDIA y BAJA.
- Barra de distribución visual (porcentaje sobre el total de tareas del tablero).

Carga por Responsable

Responsable	Tareas Asignadas	Alta	Media	Baja	Distribucion
Jorge	2	1	0	1	33%
Maria	1	1	0	0	17%
Carlos	1	0	1	0	17%
Ana	1	0	1	0	17%
Pedro	1	0	0	1	17%

3.6 Tabla de Errores Léxicos y Sintácticos

Cuando el archivo .task contiene errores, el analizador debe acumular todos los errores encontrados durante el recorrido completo del archivo (modo de recuperación de errores) y mostrarlos al finalizar. El sistema no debe detenerse ante el primer error; debe continuar el análisis para reportar todos los problemas en una sola pasada.

Cada error en la tabla debe incluir:

- Número del error: identificador incremental.
- Lexema/Token: el valor exacto que causo el error.
- Tipo de Error: Léxico o Sintáctico.
- Descripción: explicación clara del error con indicación del valor esperado.
- Línea y Columna: posición exacta del carácter que causo el error.

Ejemplo del formato de tabla de errores:

No.	Lexema	Tipo	Descripción	Línea	Col	Gravedad
1	$	Léxico	Carácter no reconocido '$'.	5	12	**ERROR**
2	"Tarea sin cerrar	Léxico	Cadena no cerrada antes del fin de línea.	8	20	**CRITICO**
3	ALTA	Sintáctico	Se esperaba ':' después de 'prioridad', se encontró 'ALTA'.	12	30	**ERROR**

3.7 Árbol de Derivación con Graphviz

Adicionalmente a los tres reportes HTML, el sistema debe generar un archivo arbol.dot con código Graphviz que represente el árbol de derivación resultante del análisis sintáctico. Este árbol muestra como las reglas de la gramática libre de contexto se aplicaron para reconocer la estructura del archivo .task.

El árbol debe incluir como mínimo:

- Un nodo raíz con el nombre de la producción inicial (<programa>).
- Nodos internos para cada producción de la gramática aplicada.
- Nodos hoja para cada token terminal reconocido.
- Los nodos terminales deben tener un color distinto a los no terminales.

Código DOT de ejemplo (parcial):

```
digraph ArbolDerivacion {
    rankdir=TB;
    node [shape=box, style=filled, fontname="Arial"];
```

```
    n0 [label="<programa>", fillcolor="#2E75B6", fontcolor=white];
    n1 [label="TABLERO", fillcolor="#D6EAF8"];
    n2 [label='"Proyecto LFP"', fillcolor="#D6EAF8"];
    n3 [label="{", fillcolor="#D6EAF8"];
    n4 [label="<columnas>", fillcolor="#D6EAF8"];
    n5 [label="<columna>", fillcolor="#D6EAF8"];
    n6 [label="COLUMNA", fillcolor="#D6EAF8"];
    n7 [label='"Por Hacer"', fillcolor="#D6EAF8"];

    n0 -> n1; n0 -> n2; n0 -> n3; n0 -> n4;
    n4 -> n5;
    n5 -> n6; n5 -> n7;
    // ... continua con el resto del árbol
}
```

3.8 Requerimientos Técnicos

- Lenguaje: C++ estándar C++17 o superior.
- Interfaz gráfica: Qt 5.15 o Qt 6.x (recomendado) o wxWidgets 3.x. Componentes mínimos: área de texto para el código fuente, tablas para tokens y errores, botones de carga y análisis, y acceso a los reportes HTML.
- El código debe organizarse en clases: Token (con enum TokenType), LexicalAnalyzer, SyntaxAnalyzer, ErrorManager, ReportGenerator y la clase principal de la GUI. Cada clase en su propio .h y .cpp.
- El proyecto debe compilar sin errores con g++ -std=c++17 o con QMake/CMakeLists.txt. El README debe incluir los comandos exactos de compilación paso a paso.
- Prohibido usar std::regex para reconocer tokens. El AFD debe implementarse en el método nextToken() con una función de transición de estados explicita.
- Prohibido usar herramientas de generación de parsers (ANTLR, Bison, Yacc, Flex, etc.). El parser debe implementarse manualmente.

3.9 Entregables

Tipo	Descripción
Código Fuente	Código completo en C++ organizado en clases y módulos, debidamente comentado. Debe incluir: clase Token, clase LexicalAnalyzer con método nextToken() basado en el AFD, clase SyntaxAnalyzer con parser descendente recursivo, clase ErrorManager, clase ReportGenerator y la clase principal de la GUI.
Manual Técnico	Documento PDF con: arquitectura del sistema (diagrama de componentes UML), diagrama de clases completo, diagrama del AFD con todos los estados y transiciones, gramática libre de contexto completa, descripción del parser y estrategia de recuperación de errores, y justificación de decisiones de diseño.
Manual de Usuario	Guia con capturas de pantalla de la GUI que explique: como compilar e instalar dependencias, estructura del archivo .task con 2 ejemplos completos, como

	cargar y analizar un archivo, como interpretar la tabla de tokens y los mensajes de error, y como abrir y navegar cada uno de los 3 reportes HTML generados.
Repositorio GitHub	Repositorio privado con estructura /src /docs /tests /examples. Commits frecuentes y descriptivos. README.md con instrucciones de compilación paso a paso, dependencias y uso. .gitignore para C++/Qt. Auxiliar agregado como colaborador antes de la entrega.
Casos de Prueba	Documento con al menos 10 casos ejecutados: entradas validas con múltiples columnas y tareas, errores léxicos (caracteres inválidos, cadenas sin cerrar), errores sintácticos (atributos faltantes, llaves sin cerrar, estructura incorrecta), múltiples errores simultáneos y casos borde. Cada caso con entrada, resultado esperado y resultado obtenido con captura.

4. Material de Apoyo

- Compilers: Principles, Techniques, and Tools (Dragon Book) -- Aho, Sethi, Ullman, Lam. 2da edicion. Capítulos 1, 2, 3 y 4.
- Introduction to the Theory of Computation -- Michael Sipser. 3ra edición. Capítulos 1 y 2.
- The C++ Programming Language -- Bjarne Stroustrup. 4ta edición.
- Documentación oficial de Qt 6: https://doc.qt.io/qt-6/
- Documentación oficial de wxWidgets: https://docs.wxwidgets.org/stable/
- Graphviz DOT Language Reference: https://graphviz.org/doc/info/lang.html
- cppreference.com -- Referencia completa de la STL de C++: https://en.cppreference.com
- Crafting Interpreters -- Robert Nystrom (especialmente Capítulos 4-6 sobre parsing).

5. Metodología

El proyecto seguirá una metodología de desarrollo incremental con cuatro fases. Se recomienda completar y verificar cada fase antes de avanzar a la siguiente, ya que cada una depende de la anterior.

Fase 1 -- Análisis del Lenguaje y Diseño (Semana 1)

- Definir formalmente la gramática léxica de TaskScript: alfabeto, tipos de tokens, patrones y errores.
- Definir la gramática libre de contexto completa del lenguaje.
- Diseñar el diagrama completo del AFD con todos sus estados y transiciones.
- Diseñar la arquitectura del sistema: clases, responsabilidades y relaciones.
- Definir los 10 casos de prueba preliminares.

Fase 2 -- Implementación del Lexer y Parser (Semana 2)

- Implementar la clase Token con el enum TokenType y sus atributos.
- Implementar la clase LexicalAnalyzer con nextToken() basado en el AFD diseñado.
- Implementar la clase SyntaxAnalyzer con el parser descendente recursivo.
- Implementar ErrorManager para acumular errores léxicos y sintácticos.
- Verificar el funcionamiento desde consola con todos los casos de prueba.

Fase 3 -- Integración con GUI y Reportes (Semana 3)

- Diseñar e implementar la GUI en Qt o wxWidgets con todos los paneles requeridos.
- Integrar el lexer y parser con la GUI para carga de archivos y visualización de resultados.
- Implementar ReportGenerator para los 3 reportes HTML y el código DOT de Graphviz.

Fase 4 -- Pruebas, Corrección y Documentación (Semana 4)

- Ejecutar los 10 casos de prueba y documentar los resultados con capturas de pantalla.
- Corregir errores y verificar la cobertura del parser con casos borde.
- Redactar el Manual Técnico con todos los diagramas requeridos.
- Redactar el Manual de Usuario con capturas de los 3 reportes.
- Organizar el repositorio GitHub y verificar que el proyecto compila desde cero siguiendo el README.

6. Recursos y Herramientas a Utilizar

Entorno de desarrollo:

- IDE: Qt Creator (recomendado), CLion, Visual Studio 2022 o VS Code con extensión C/C++.
- Compilador: g++ con soporte C++17 (MinGW en Windows, GCC en Linux/macOS) o MSVC 2019/2022.
- Sistema de construcción: QMake (.pro) o CMake 3.16+.

Restricciones técnicas:

- Lenguaje: C++ (C++17 o superior). Interfaz: Qt 5/6 o wxWidgets 3.x.
- Tokenizacion únicamente con funciones nativas de std::string. Prohibido std::regex.
- Parser implementado manualmente. Prohibido ANTLR, Bison, Yacc, Flex o similares.

7. Desarrollo de Habilidades Blandas

Autogestión del tiempo

Se recomienda 6-8 horas semanales. La Fase 2 (parser) es la más demandante ya que requiere comprender la gramática y traducirla a código. Los ciclos de depuración de errores sintácticos pueden ser extensos.

Responsabilidad y compromiso

El código, los diagramas y la documentación deben ser resultado del trabajo personal e individual. La prohibición de herramientas automáticas obliga al estudiante a comprender a fondo cada producción de la gramática y cada función del parser.

Resolución de problemas

El estudiante deberá resolver ambigüedades en la gramática, implementar recuperación de errores sintácticos robusta, y manejar casos bordes como archivos vacíos, bloques sin tareas y atributos faltantes.

Pensamiento critico

El estudiante debe evaluar múltiples estrategias para implementar el parser y justificar sus decisiones en el manual técnico con criterios de eficiencia, mantenibilidad y legibilidad del código.

8. Cronograma

El cronograma describe las etapas clave del proyecto y el proceso de asignación, elaboración y calificación.

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto		
Elaboración		
Calificación		

9. Rubrica de Calificación

9.1 Requisitos para optar a la calificación

Tema	Descripción	Cumple (Si/No)
Cumplimiento de tecnología	El proyecto debe desarrollarse en C++ (C++17 o superior) con interfaz gráfica funcional en Qt 5/6 o wxWidgets. No se permite el uso de std::regex ni herramientas de generación de parsers.	

Uso de herramientas requeridas	Repositorio en GitHub con commits organizados, README con instrucciones de compilación, estructura de carpetas definida y .gitignore apropiado.	
Gestión y entregas	Entrega puntual del link al repositorio en UEDI con todos los entregables: código compilable, manuales en PDF y casos de prueba documentados.	
Documentación obligatoria	Manual Técnico con diagrama de clases UML, diagrama del AFD, gramática libre de contexto y descripción del parser. Manual de Usuario con capturas de la GUI y los 3 reportes. Ambos en PDF.	
Funcionalidad mínima	El sistema debe: cargar archivos .task, tokenizar y parsear el lenguaje TaskScript, generar tabla de tokens y errores, generar los 3 reportes HTML y el árbol de derivación Graphviz.	

9.2 Resumen de Puntuaciones

Categoría	Punteo	Obtenido
Habilidades (Doc. Técnica, Manual de Usuario, Repositorio, Presentación, Casos de Prueba)	**40 pts**	
Conocimientos (AFD, Parser, Tablas, GUI, 3 Reportes HTML + Graphviz, Opcionales)	**60 pts**	
TOTAL	**100 pts**	

9.3 Rubrica Detallada

1	**Habilidades**	**40**			
1.1	**Documentación Técnica**	10	10-6	Manual técnico completo: arquitectura UML, diagrama de clases, diagrama del AFD, GLC completa, descripción del parser y justificación de decisiones.	Manual incompleto: faltan diagramas UML, la GLC no está documentada, parser sin describir.
1.2	**Documentación de Usuario**	8	8-5	Manual completo con instrucciones de compilación, capturas de la GUI, ejemplos de archivos .task e interpretación de los 3 reportes.	Instrucciones confusas, faltan capturas o no permite operar la aplicación de forma autónoma.
1.3	**Repositorio GitHub**	7	7-4	Repositorio organizado: /src /docs /tests /examples, commits frecuentes y descriptivos, README con instrucciones de compilación.	Repositorio desorganizado, pocos commits o genéricos, README ausente.
1.4	**Presentación**	8	8-5	Presentación profesional: contexto del problema, arquitectura, demostración en vivo con GUI, explicación del parser y conclusiones técnicas.	Sin demostración funcional o el estudiante no puede explicar el parser ni las decisiones de diseño.
1.5	**Casos de Prueba**	7	7-4	Al menos 10 casos ejecutados: validos, errores léxicos, errores sintácticos, múltiples errores y casos borde.	Menos de 6 casos, mal documentados o sin evidencia de ejecución.

2	Conocimientos	60			
2.1	**Diseño e Implementación del AFD**	10	10-6	AFD implementado manualmente sin std::regex. Reconoce todos los tokens TaskScript. Tracking preciso de línea y columna.	Usa std::regex, estados faltantes o incorrectos, tracking impreciso.
2.2	**Parser Descendente Recursivo**	15	15-9	Parser implementado manualmente con funciones para cada producción de la GLC. Construye árbol de derivación. Recuperación de errores sintácticos.	Usa herramientas automáticas, producciones faltantes, no construye árbol o no recupera errores.
2.3	**Tablas de Tokens y Errores**	8	8-5	Tabla de tokens con precisión 100%. Tabla de errores con tipo (léxico/sintáctico), descripción y posición. Ambas en GUI y HTML.	Tablas incompletas, posición incorrecta o no se generan ambas tablas.
2.4	**Interfaz Gráfica (GUI)**	8	8-5	GUI funcional en Qt o wxWidgets con todos los paneles requeridos. Navegación fluida.	GUI incompleta, botones no funcionales o diseño confuso.
2.5	**3 reportes HTML y Graphviz**	15	15-9	Genera los 3 reportes HTML requeridos con el formato descrito. El código DOT produce árbol de derivación valido.	Genera menos de 2 reportes, formato incompleto o el DOT no produce salida valida.
2.6	**Funcionalidades Opcionales**	4	4-2	Implementa al menos una funcionalidad opcional correctamente.	No implementa opcionales o están incompletas.

10. Entrega

- Subir el link al repositorio de GitHub en el apartado de UEDI antes de la fecha y hora límite.

El repositorio debe contener:

- /src -- Código fuente en C++ y archivos .pro o CMakeLists.txt
- /docs -- Manual Técnico (PDF) y Manual de Usuario (PDF)
- /tests -- Archivos .task de casos de prueba (mínimo 10, validos e inválidos)
- /examples -- Al menos 2 archivos .task completos de ejemplo
- README.md -- Instrucciones de compilación, dependencias y uso
- .gitignore -- Para proyectos C++/Qt

- Nombre del repositorio: LFP_1S2026_#Carnet. Carpeta interna: Proyecto2.
- Agregar al auxiliar de la sección como colaborador antes de la entrega.
- La calificación es PRESENCIAL. El estudiante debe demostrar la GUI en funcionamiento, cargar un archivo .task en tiempo real y explicar el diseño del parser y la gramática.

Fecha límite: Según indicación del auxiliar, antes de las 23:59 hrs. No se reciben entregas fuera del plazo.

EN CASO DE ENCONTRAR COPIAS PARCIALES O TOTALES DEL CODIGO O DOCUMENTACION, SE ANULARÁ EL PROYECTO AUTOMATICAMENTE, SE NOTIFICARÁ AL CATEDRATICO Y SE REPORTARÁ A LA ESCUELA DE CIENCIAS Y SISTEMAS PARA LAS SANCIONES CORRESPONDIENTES.